

Guia practica: Docker y despliegue web

Proyecto TeoNova-Vet / Veterinaria POS

Recomendacion corta: No es obligatorio aprender Docker para publicar la app, pero para este proyecto si es muy recomendable probarlo en Docker antes de subirlo. Te permite ensayar frontend, backend y MySQL como si estuvieran en un servidor real.

1. Que es Docker en palabras simples

Docker sirve para empackar una aplicacion con todo lo que necesita para ejecutarse. En vez de decir "en mi computador funciona", Docker ayuda a que funcione igual en otro computador o servidor.

- Imagen: una plantilla lista para crear una app o servicio.
- Contenedor: una instancia corriendo de esa imagen.
- Docker Compose: un archivo que levanta varios contenedores juntos.
- Volumen: almacenamiento persistente para que los datos no se borren al reiniciar contenedores.

2. Como aplica a este proyecto

Frontend	React + Vite. Se compila con npm run build y genera archivos estaticos en dist.
Backend	NestJS. Se compila con npm run build y arranca con node dist/main.
Base de datos	MySQL. El esquema esta en la carpeta database.
API	El frontend llama a /api, por eso en produccion conviene usar un proxy hacia el backend.

3. Que debes instalar en Windows

1. Instala Docker Desktop para Windows desde el sitio oficial de Docker.
2. Permite usar WSL 2 si Windows lo solicita.
3. Reinicia el computador si el instalador lo pide.
4. Abre Docker Desktop y espera a que diga que esta corriendo.
5. Abre PowerShell y verifica la instalacion.

```
docker --version
docker compose version
```

4. Comandos basicos que vas a usar

docker ps	Ver contenedores corriendo.
docker ps -a	Ver todos los contenedores, incluso detenidos.
docker compose up	Levantar los servicios definidos en docker-compose.yml.
docker compose up --build	Reconstruir imagenes y levantar servicios.
docker compose down	Detener y eliminar contenedores de ese proyecto.
docker compose logs -f backend	Ver logs del backend en vivo.

5. Ruta recomendada para tu app

1. Confirma que backend y frontend compilan localmente.
2. Crea Dockerfile para backend.
3. Crea Dockerfile para frontend con Nginx.
4. Crea docker-compose.yml con mysql, backend y frontend.
5. Carga la base de datos usando database/schema.sql o la migracion correcta.
6. Prueba login, ventas, inventario, clientes, reportes y exportacion Excel.
7. Cuando todo funcione en Docker local, subelo a un VPS con Docker.

6. Variables importantes de produccion

NODE_ENV	production
PORT	3001 o el puerto interno del backend
DB_HOST	mysql si usas docker-compose, o host de la base si es externa
DB_NAME	veterinaria_pos u otro nombre elegido
DB_USERNAME	usuario de MySQL
DB_PASSWORD	contrasena segura
JWT_SECRET	cadena larga, aleatoria y secreta
CORS_ORIGINS	dominio real, por ejemplo https://tudominio.com

TZ	America/Bogota
DB_TIMEZONE	-05:00

7. Checklist antes de subir a internet

- ☐ El backend compila sin errores.
- ☐ El frontend compila sin errores.
- ☐ El backend arranca con NODE_ENV=production.
- ☐ La base de datos esta creada y tiene tablas.
- ☐ El usuario administrador existe.
- ☐ JWT_SECRET no es el valor de ejemplo.
- ☐ CORS_ORIGINS tiene el dominio correcto.
- ☐ El frontend puede entrar a /api sin error de CORS.
- ☐ Hay respaldo de la base de datos.
- ☐ El dominio tiene HTTPS activo.

8. Donde hospedarlo

Para este proyecto, mi recomendacion principal es un VPS con Docker. No es la unica opcion, pero es la mas clara para controlar backend, frontend y MySQL sin depender de paneles limitados.

VPS con Docker	Recomendado. Bueno para Hostinger VPS, DigitalOcean, Contabo, Hetzner o AWS Lightsail.
Railway / Render	Mas simple al inicio, pero revisa costos y soporte de MySQL.
Hosting compartido	Solo si soporta Node.js de verdad. Puede ser incomodo para NestJS.
Vercel/Netlify	Sirven para frontend, pero necesitarias backend y base de datos aparte.

9. Plan de aprendizaje rapido

1. Dia 1: instalar Docker Desktop y correr docker --version.
2. Dia 2: levantar un MySQL de prueba con Docker.
3. Dia 3: crear Dockerfile del backend.
4. Dia 4: crear Dockerfile del frontend.
5. Dia 5: unir todo con docker-compose.

6. Dia 6: probar flujo completo de la app.

7. Dia 7: preparar VPS y subir primera version.

10. Siguiete paso recomendado

El siguiente paso concreto seria crear los archivos Dockerfile y docker-compose.yml dentro del proyecto. Con eso podras ejecutar un solo comando para levantar la app completa:

```
docker compose up --build
```